

■ Tweet Sentiment Analyzer

Sentiment Analysis on the Sentiment140 Dataset (1.6M Tweets)

1. Project Overview

This project builds an end-to-end sentiment analysis pipeline that classifies tweets as **Positive** or **Negative** using classical machine learning. It covers the full journey from raw, noisy tweet data to a deployed, interactive web application where a user can type any text and get an instant sentiment prediction.

2. Dataset

Source: Sentiment140 dataset (Go, Bhayani & Huang, 2009, Stanford University) — 1.6 million tweets collected via the Twitter API and automatically labeled using emoticons as noisy sentiment signals.

Columns: target, id, date, query, user, text — where target is 0 (negative) or 4 (positive).

Note: the training portion of Sentiment140 contains only positive and negative labels; the neutral class (2) appears only in Stanford's small, separate hand-labeled test set.

3. Project Pipeline

- 1 **Data Preprocessing:** Removed URLs, @mentions, hashtag symbols (keeping the hashtag word), non-alphabetic characters, and extra whitespace; lowercased all text.
- 2 **Exploratory Data Analysis (EDA):** Analyzed class distribution, tweet length distribution, most frequent words per sentiment class, and generated word clouds for positive vs. negative tweets.
- 3 **Feature Engineering:** Converted cleaned text into numeric vectors using TF-IDF (unigrams + bigrams, max 50,000 features, min document frequency 5).
- 4 **Train/Test Split:** Split data 80/20 before vectorizing, stratified on sentiment label, to avoid leaking test vocabulary into training.
- 5 **Model Development:** Trained and compared three classifiers on the same split: Logistic Regression, Multinomial Naive Bayes, and Random Forest (on a stratified subsample for tractability).
- 6 **Model Selection:** Selected the best model by F1 score; saved the model and the fitted TF-IDF vectorizer using joblib for reuse without retraining.
- 7 **Deployment:** Built a Streamlit web app (app.py) that loads the saved model and vectorizer to classify any user-entered text in real time, then deployed it and pushed the project to GitHub.

4. Model Comparison

Three models were evaluated on the same held-out test set using Accuracy, Precision, Recall, and F1-score. Logistic Regression and Naive Bayes were trained on the full training set; Random Forest was trained on a stratified subsample for computational tractability, but evaluated on the full, untouched test set for a fair comparison.

Model	Trained On	Notes
Logistic Regression	Full training set	Strong, fast, well-calibrated baseline for sparse text features
Multinomial Naive Bayes	Full training set	Classic, very fast text classifier
Random Forest	Stratified subsample	Doesn't scale well to millions of sparse TF-IDF features

The best-performing model (by F1 score) was saved as the final production model.

5. Tech Stack

- Python — core language
- Pandas / NumPy — data handling
- Scikit-learn — TF-IDF vectorization, Logistic Regression, Naive Bayes, Random Forest, evaluation metrics
- Matplotlib / Seaborn / WordCloud — EDA visualizations
- Streamlit — interactive web app for real-time predictions
- Joblib — model and vectorizer persistence
- Git / GitHub — version control and project hosting
- Jupyter Notebook — end-to-end analysis and experimentation

6. Application Architecture

The Streamlit app (`app.py`) loads the saved `sentiment_model.joblib` and `tfidf_vectorizer.joblib` files at startup. When a user enters text, the app applies the same cleaning function used during training, transforms the cleaned text with the saved TF-IDF vectorizer, and passes it to the model to produce a Positive/Negative prediction, along with a confidence score where available.

Running Locally

```
pip install -r requirements.txt
streamlit run app.py
```

7. Repository Contents

File	Description
<code>Sentiment_Analysis_Sentiment140.ipynb</code>	Full notebook: EDA, preprocessing, feature engineering, model training/comparison
<code>app.py</code>	Streamlit app serving real-time sentiment predictions
<code>sentiment_model.joblib</code>	Best-performing trained model

<code>tfidf_vectorizer.joblib</code>	Fitted TF-IDF vectorizer
<code>requirements.txt</code>	Python dependencies
<code>README.md</code>	Project documentation

8. Future Enhancements

- Compare against deep learning approaches (e.g. a fine-tuned DistilBERT model)
- Add a neutral sentiment class using additional labeled data
- Expose predictions via a REST API (e.g. FastAPI) in addition to the Streamlit UI
- Add batch prediction support (upload a CSV of tweets, get sentiment for all of them)

9. Dataset Citation

Go, A., Bhayani, R. and Huang, L., 2009. *Twitter sentiment classification using distant supervision*. CS224N project report, Stanford, 1(2009), p.12.